

Project-Side Integration + OTA-Upload CLI — To-Be Design (research + planning)

Revision: 1 Last modified: 2026-07-10T11:18:54Z

Design proposal only. This document specifies a production **project-integration CLI** (`helix-ota`) that a consuming project's CI/CD uses to authenticate with **one account + one project under it** and publish OTA updates. It **extends the mechanisms that already exist** (per `11_existing_upload_and_device_update.md`, the authoritative current-state audit — cited, never contradicted) rather than reinventing them. Account / user / membership / API-key **entity shapes are owned by** `20_target_multitenancy_data_model.md` (SSOT); this doc *consumes* those shapes and never redefines a conflicting one — where a shape is load-bearing here it is cited to the one concrete in-repo sketch (`001_initial_schema.up.sql`) and otherwise deferred to `20_*`. No code is written under this directive; the plan is operator-gated (`00_INDEX.md` §2). Every “as-is” fact carries a `file:line` from the grounded audits; every proposal is a **recommendation with tradeoffs**, never a silent decision (§11.4.66 / §11.4.6).

0. Current-state anchor (what we **EXTEND**, from `11_*` + `10_*`)

The extension target, verbatim from the grounded audits (do not contradict):

- **Upload/publish EXISTS and is real.** `POST /artifacts/upload` runs a real S1–S6 multipart validation pipeline (`handlers_artifact.go:47; validate :152-173`); `POST /releases` (`handlers_release.go:16`) and `POST /deployments` (`handlers_deployment.go:28`) **publish/deploy**. Every write is bearer-authenticated + audited (`server.go:189-190`).

- **None of it is project- or account-scoped.** `store.Artifact/Release/Deployment/ Device` carry no `ProjectID/account` field (`store.go:58-99, :35-56`); `token Claims{sub,roles,iat,exp}` carry no tenant dimension (`token.go:31-36`); the `Project+ProjectAccess` ACL exists but is **not wired** into the OTA data model (`11_* §1, §7; 10_* §4`).
- **The only upload client is the ota-manager SPA.** `useUploadArtifact` → `apiMultipartPost('/artifacts/upload', formData)` with a `Authorization: Bearer token` and **no project header/query/path** (`use-artifacts.ts:35-51; api-client.ts:28-36`); its **project switcher is a MOCK** (`project-switcher.tsx:13-19`). **There is no upload CLI/SDK** (`11_* §2`).
- **Artifact bytes are validated then discarded.** `StorageRef` is a synthesized placeholder `s3://helix-artifacts/<id>` (`handlers_artifact.go:184`); only metadata + the verified signature persist. **No real object storage is in-repo** (`11_* §Honest-gaps 1`). There is also **no list-artifacts endpoint** (`useUploadArtifact.ts:4-7`).
- **Signing key is a single global.** `resolvePublicKey` (`handlers_artifact.go:283-288`) takes the `ed25519` verify key ONLY from `s.pubKey`, loaded from one global `HELIX_ARTIFACT_PUBKEY` (`config.go:72-77, server.go:109-112`) — never from the request (trust boundary, matches project `CLAUDE.md`).
- **A login flow already exists.** `POST /api/v1/auth/login` → `handleLogin` exchanges `username+password` for an `access+refresh` pair (`handlers_auth.go:77-99, issueTokenPair :123-137`); refresh tokens are opaque, single-use-rotating, TTL-bounded.
- **An api_keys shape already exists ON PAPER** (design-target migration, not shipped): `api_keys(id, user_id, key_hash, name, permissions JSONB, expires_at, revoked_at, created_at)` — hash-only, cleartext shown once (`docs/research/main_specs/1.0.0-mvp/database/migrations/001_initial_schema.up.sql:57-71`), plus a `users` table (`:39-51`). Neither is in the shipped store (`10_* §1, §3`).

This CLI is therefore a **new client binary + a set of additive server seams**, not a rewrite. The four hardest dependencies it exposes (`account/project` scope columns, a tenant-aware token, a per-account key registry, and **real object storage**) are named explicitly below as dependencies/risks.

1. Auth model — non-interactive CI/CD authentication

1.1 The credential: an account + project-scoped API key

A CI pipeline cannot type a password. The credential **MUST** be a **named, revocable, long-lived API key bound to exactly one account and (optionally) one project**, issued by a super-admin (there is no self-registration — 00_INDEX.md §1.2), presented non-interactively.

The credential shape **extends the existing api_keys sketch** (SSOT: 20_*; grounded sketch: 001_initial_schema.up.sql:57-71). The extension 20_* must define is the two scope columns that sketch lacks:

Column	Source	Purpose
id, key_hash, name, permissions, expires_at, revoked_at, created_at	already sketched (001_initial_schema.up.sql:57-71)	hash-only storage, cleartext shown ONCE at creation, named + revocable + expirable
account_id	20_* owns this shape	binds the key to exactly ONE account (tenant)
project_id (nullable)	20_* owns this shape	optional narrowing to ONE project; NULL = “any project in the account the caller is authorized for”
user_id / owner	already sketched (:59), or an account-owned “robot” subject (20_*)	attributes the key for audit (10_* §6 UserID)

Only `key_hash` is stored server-side; the cleartext key (recommended opaque form `helixk_<account-short>_<random>`, high-entropy) is shown once and never persisted in cleartext (matches the sketch's `key_hash` + the industry pattern in §Sources). This respects the established trust boundary: **scope is authoritative from the stored key row, never self-asserted by the request** (the same rule as `resolvePublicKey` and `TrustTLSProxy`, 10_* §7).

1.2 Where the client stores it (§11.4.10 no-leak)

Resolution order on the client, highest precedence first — matching the CI-token norm across Mender/Expo/balena (§Sources):

1. **--token <value> flag** — explicit, for one-off invocations (avoid in shared shells).
2. **HELIX_OTA_TOKEN env var** — the primary CI path (GitHub Actions / GitLab secret → env), same shape as Expo's `EXPO_TOKEN` and Mender's `--token-value $PAT`.
3. **Credential file** `~/.config/helix-ota/credentials` (or `$HELIX_OTA_HOME`), one service per file, `chmod 600`, parent dir `chmod 700` (§11.4.10). Written only by `helix-ota login --token`.
4. **OS keyring** (optional, opt-in) — Secret Service / macOS Keychain / Windows Credential Manager for developer laptops.

Hard §11.4.10 rules the CLI enforces: the token is **never** printed or logged (redacted to `helixk_...<last4>` in any diagnostic), **never** written to a world-readable path, **never** committed — the CLI ships a `.gitignore` fragment (`.helix-ota/`, `*.helix-ota-credentials`) and a `.env.example` (§11.4.77) for consuming projects; a `helix-ota doctor` subcommand audits for an accidentally-tracked token before first push.

1.3 How the key maps to (account, project) scope on the server — two options

The token today has **no tenant dimension** (`token.go:31-36`), so the key → scope mapping is a genuine design fork. Both options consume `20_*`'s `account/project/key` shapes; the choice is a hot-path decision `21_authz_rbac_superadmin.md` finalizes.

Option A — static key sent directly as Bearer on every request; server resolves key → scope per call. Server middleware hashes the presented key, looks up the `api_keys` row, and derives (`account_id`, `project_id`, `permissions`) for that request; enforcement reuses the

requireProjectAccess pattern (handlers_project.go:37-74) plus a new requireAccountAccess. - *Pros*: simplest CLI; one credential; matches balena's named API-key model (balena login --token, then Bearer). Revocation is immediate (row lookup each call). - *Cons*: a long-lived secret rides the wire on **every** request; a per-call hash+DB lookup on the hot path; the standing exposure window is the key's whole lifetime.

Option B — key is an EXCHANGE credential (recommended). helix-ota login --token \$KEY calls a new exchange endpoint (e.g. POST /auth/token-exchange, sibling of handleLogin) that validates the key and returns a **short-lived access token carrying account_id+project_id+role claims** — i.e. issueTokenPair (handlers_auth.go:123) extended with the tenant claim 21_* adds to Claims (token.go:31-36). Every subsequent request uses that scoped bearer on the existing authenticated hot path. - *Pros*: the long-lived key touches the wire **once per session**; the hot path is the already-shipped bearer+Claims flow (minimal new surface); least standing exposure (§11.4.10); short-lived tokens auto-expire; mirrors the existing login/refresh design and Expo robot-account exchange. - *Cons*: one extra round trip at session start; **requires** the tenant claim in Claims (a 21_* dependency); revocation of the long-lived key doesn't retroactively kill an already-minted short-lived token (bounded by its short TTL — 15 min default, config.go:54-55).

Recommendation: Option B, because it reuses the existing login/token machinery, keeps the long-lived secret off the per-request wire, and inherits the short-TTL auto-expiry the anti-leak posture wants. Adopt Option A only if 21_* decides the tenant claim is out of scope for the first milestone, in which case direct-key Bearer is an acceptable, revocation-immediate interim.

2. Command surface

helix-ota is a single static Go binary (natural home: server/cmd/helix-ota/, beside ota-server/ota-device-emu, 11_* §2), reusing the server's wire.go request structs so the CLI and server never drift. It carries a **local context** (server URL + account + project + os/target defaults) like kubectl, so a project-scoped key need not re-state --project on every call.

Command	Maps to endpoint (existing / +new scoping)	Notes
helix-ota login --token \$KEY / login -u -p	POST /auth/token-exchange (new, Option B) or POST / auth/login (handlers_auth.go:77)	stores scoped session; interactive user/pass path reuses handleLogin
helix-ota logout	local session clear	removes cached short- lived token
helix-ota whoami	resolve from token/key	prints resolved (account, project, role) — proves scope, no bluff
helix-ota config set- context -- server -- account -- project --os -- target-model	local only	kubectl-style context; no server call
helix-ota upload <artifact.zip> --project <p> [--account <a>] [--version --os --target-model --sha256 -- signature]	POST /artifacts/upload (handlers_artifact.go:47) + (account,project) association	multipart file+metadata(+sha256/ signature); the core command (§3)
helix-ota artifacts list --project <p>	MISSING endpoint (useUploadArtifact.ts:4-7)	dependency: a list- artifacts endpoint must be added (§4)
helix-ota release create --artifact <id> --project <p>	POST /releases (handlers_release.go:16) + project scope	referenced artifact must be Verified + version strictly- monotonic (:43-69)
helix-ota release list -- project <p>	GET /releases (server.go:206)	scoped list
helix-ota deploy create --release <id>	POST /deployments (handlers_deployment.go:28) + project scope	MVP accepts only all- targets (:39-43)

Command	Maps to endpoint (existing / +new scoping)	Notes
--strategy all- targets -- project <p>		
helix-ota status <deploymentId> / deploy list	GET /deployments/:id (server.go:210)	progress is derived server-side from telemetry (deriveProgress, 11_* \$5)
helix-ota doctor	local + GET /health	token-leak audit, config sanity, server reachability

Global flags: --server, --project, --account, --json (machine-readable output for CI), --token, -v (redacted-logging). Exit codes are stable + documented for CI gating (0 ok / non-zero per failure class).

3. Upload flow (the core path)

1. **Client** builds multipart/form-data exactly as the server expects (file payload bytes + metadata JSON ArtifactUploadMetadata + optional sha256 + signature), mirroring useUploadArtifact (use-artifacts.ts:35-51) and the wire struct (wire.go / api.ts:723-735), and sends POST /artifacts/upload with the scoped bearer (Option B token or Option A key) — plus the caller’s chosen --project.
2. **Server associates the artifact with (account, project).** This needs the currently-absent scope columns on store.Artifact (11_* \$1: no ProjectID today). The rule, preserving the trust boundary: the **account is taken from the resolved token/key scope, never from a request field** (a client cannot upload into an account its key does not grant); the **project** is the caller-supplied --project, which the server **validates against the key’s authorized project set** (requireProjectAccess pattern, handlers_project.go:37-74) and rejects (403) if out of scope. The accepted store.Artifact (built at handlers_artifact.go:176-198) gains account_id + project_id (shapes owned by 20_*).
3. **Monotonicity + “latest” become per-scope.** LatestRelease(os, target_model) is global today (10_* \$8) and would let one tenant’s version block another’s; it must become LatestRelease(account / project, os, target_model). Same for

ActiveDeploymentForTarget(os, model, group) (deployment uniqueness) and project/ device/group name uniqueness (10_* \$8 flags each). Flagged as a 20_*/22_* scoping task the CLI depends on but does not itself implement.

4. **Signing/verify against a per-account key.** Today resolvePublicKey (handlers_artifact.go:283-288) returns one global HELIX_ARTIFACT_PUBKEY. Extend it to a **per-account key registry** keyed on the resolved account_id, with the global key as fallback for un-migrated accounts. **The trust boundary is unchanged:** the verify key still comes only from server config/registry, never from the request — only the *lookup key* gains an account dimension. The CLI never supplies a verify key; it supplies only the detached signature over payload.bin (S3, 11_* \$3).
 - *Recommendation:* per-account ed25519 key in a server-side registry (config- or KMS-backed), global fallback during migration. *Tradeoff:* per-account key rotation + a registry lookup vs. the simplicity of one global key; a shared global key means any account's signer can sign for the platform — unacceptable long-term for tenant isolation, so per-account is the target, staged behind the fallback.
 5. **Response + evidence.** The CLI surfaces the returned Artifact (id, Verified status, stored signature) as --json; whoami/status let CI assert the artifact landed under the right (account, project) — the anti-bluff hook for \$6.
-

4. Object-storage gap (honest dependency / risk)

This is the load-bearing production dependency and it is real, not a design nicety (11_* \$Honest-gaps 1). Today handleUploadArtifact validates the uploaded bytes with S1–S6 and then **discards them**; StorageRef is a synthesized placeholder s3://helix-artifacts/<id> (handlers_artifact.go:184) and the device's download URL is ArtifactBaseURL/<id>.zip (handlers_client.go:232-234) pointing at an external “Storage brick” **that is not present in this repo**. Consequence: the CLI can authenticate, validate, and register an artifact today, **but the platform cannot durably store or serve the payload bytes** — so an end-to-end “upload → device downloads → device applies” is not yet possible from in-repo code alone.

What a production upload requires (dependency the CLI blocks on, not owned by this doc):

1. **A real object store behind the accept path.** After S1–S6 accept, stream the payload bytes to a durable store (S3 / GCS / MinIO), persist the **real** StorageRef (replacing the placeholder at `handlers_artifact.go:184`), and resolve the device download URL from it (signed, time-bounded URL).
2. **Per-account/project key prefix or bucket** so tenant A's bytes are physically isolated from tenant B's (composes with the §3 scoping + 26_security_threat_model.md).
3. **A storage interface seam** (like `store.Repository`) so dev uses rootless-podman **MinIO** (§11.4.161 via the `containers` submodule) and prod uses S3/GCS — no ad-hoc client in handlers.
4. **A list-artifacts endpoint** (`GET /artifacts?project=...`) — absent today (`useUploadArtifact.ts:4-7`) — for helix-ota artifacts list and the SPA's release picker.

Recommendation: introduce the Storage seam + MinIO dev backend as a **prerequisite milestone** in `30_delivery_plan.md`, gating the CLI's e2e claim; per-account prefixing; signed download URLs. *Tradeoff:* a self-hosted MinIO adds an infra component and a dev-boot dependency, but it is the only way to produce the §6 e2e captured evidence honestly (a “PASS” over discarded bytes would be a §11.4 bluff). **Until object storage lands, the CLI's upload+register+release+deploy path is validatable, but “device downloads and applies the exact bytes” is not — this is stated as fact, not deferred silently (§11.4.6).**

5. Incorporation into an existing consuming project (minimal recipe)

A consuming project adds Helix OTA publishing in five steps. (Note: **CI/CD automation is DISABLED in *this* repo per §11.4.156** — the snippets below are for the *consumer's* own pipeline; no active `.yaml` workflow is added to `helix_ota`.)

1. **Provision (super-admin side, one-time).** The platform super-admin creates the account, the project under it, and issues a scoped API key (`helix-ota` admin flow or the super-admin console, 21_*/23_*). The cleartext key is handed to the project owner once.

2. **Store the key as a CI secret.** GitHub Actions repo secret / GitLab CI masked variable / Vault — surfaced to the job as `HELIX_OTA_TOKEN`. **Never** in git (§11.4.10); the project's `.gitignore` includes `.helix-ota/`.

3. **Commit non-secret config only.** `.helix-ota/config.yaml` (server URL, account, project, os, `target_model` defaults) is safe to commit; the credential is not. Ship a `.helix-ota/config.example.yaml` + `.env.example` documenting `HELIX_OTA_TOKEN` (§11.4.77 regeneration/README).

4. **Add the publish step** after the OTA artifact is built and signed:

```
# consumer project's own pipeline (illustrative; not added to
helix_ota, §11.4.156)
- name: Publish OTA update
  env:
    HELIX_OTA_TOKEN: ${ secrets.HELIX_OTA_TOKEN }
  run: |
    helix-ota upload dist/update.zip \
      --project myproj --version "${VERSION}" --os android --
      target-model rk3588
    helix-ota release create --artifact "$(helix-ota artifacts
      list --project myproj --json | jq -r '.[0].id')" --
      project myproj
    helix-ota deploy create --release "${RELEASE_ID}" --
      strategy all-targets --project myproj
```

5. **Verify in CI.** `helix-ota whoami` (proves scope) + `helix-ota status <deploymentId> --json` (proves the deployment exists) gate the job; a non-zero exit fails the pipeline. (`artifacts list` depends on the §4 endpoint gap being closed.)

Recommendation: ship the CLI as a downloadable static binary + a thin `install.sh` and a container image (rootless-podman-runnable) so consumers pin a version; document the minimal `config.yaml` in the CLI's own `docs/scripts/` (§11.4.18). *Tradeoff:* a container image is heavier than a bare binary but removes host-toolchain assumptions in locked-down CI.

6. Test strategy for the CLI (anti-bluff, no mock beyond unit — §11.4.27 / §11.4.69)

Every layer below the unit layer exercises the **real** server + **real** store; mocks live only in unit tests (§11.4.27). Each PASS cites captured evidence under `docs/qa/<run-id>/` (§11.4.83) via `ab_pass_with_evidence` (§11.4.69) — never a metadata-only/absence-of-error PASS.

- **Unit** (mocks allowed here only): arg/flag parsing, context resolution + precedence (`--token > env > file > keyring`), **credential redaction** (assert the raw token never appears in any log line — the §11.4.10 guard), multipart body construction, retry/backoff, exit-code mapping. Table-driven.
- **Integration** (real server, no mock): boot the real `ota-server` (`httptest` or a real binary) with a real (memory or Postgres) store; drive `login → upload → release create → deploy create` with a real scoped key. Assert: (a) the artifact row persists with the correct (`account_id, project_id`); (b) a **cross-project key is rejected 403** on upload/release/deploy (tenant isolation — negative test, 26_*); (c) a cross-account key cannot read/list another account's artifacts. Evidence: captured server JSON responses + store-row dumps.
- **E2E** (real everything): `auth → upload → release → deploy → device-verify`, against a real server + **real object storage (MinIO)** + the Go device emulator (`ota-device-emu`, 11_* §4) polling `GET /client/update`. Assert the emulator receives the offer for *this* account/project's release, **downloads the exact bytes**, and `VerifyBeforeApply` accepts them (SHA-256 + signature, 11_* §3). Sink-side captured evidence (§11.4.69 `cast/download` taxonomy): the emulator's received manifest
 - the persisted real `StorageRef` + the downloaded+verified artifact hash. **Blocked on §4 object storage** — until then this layer is honestly `SKIP-with-reason: object_storage_absent`, never a fake PASS.
- **Security / tenant isolation**: the cross-account/cross-project negative matrix above as a first-class suite (26_*); token-leak scan; expired/revoked-key rejection.
- **Stress + chaos** (§11.4.85): N concurrent uploads across accounts (no cross-tenant bleed, no deadlock); kill the CLI mid-upload / corrupt the token / drop the network mid request — the CLI fails cleanly with a categorized non-zero exit, never a partial registered artifact.
- **Full-automation / re-runability** (§11.4.98): every layer is self-driving end to end (credential bootstrapped once from env, no

human keystroke mid-run) and passes at -count=3 with self-cleaning state.

- **HelixQA Challenge** (§11.4.27 layer 4): a bank entry driving the real CLI → server → device journey with captured evidence.

Recommendation: gate the CLI's "done" on the integration + security layers GREEN with captured evidence now, and mark the e2e layer explicitly blocked on the object-storage milestone (§4) so the coverage ledger is honest rather than green-over-a-gap.

Sources verified 2026-07-10

External research per §11.4.8 / §11.4.99 — production CLI-auth-for-CI patterns and OTA/release-publishing CLIs that model project/fleet-scoped upload tokens. Findings corroborate the design: scoped, revocable, hash-stored, least-privilege tokens delivered via env var, exchanged/used non-interactively, with an upload command that both uploads bytes and creates a release entry.

- **Mender — Using the APIs / CI-CD** (Personal Access Tokens as API keys for CI/CD; `mender-cli artifacts --server ... --token-value $PAT upload release.mender`; requires Deployments-Manager + Releases-Manager roles; Direct Upload streams bytes to artifact storage): <https://docs.mender.io/server-integration/using-the-apis> + <https://docs.mender.io/artifact-creation/ci-cd> — validates §1 (scoped PAT from CI), §2 (upload command), §3 (role-scoped publish), §4 (direct-to-object-storage upload).
- **balena — Deploy to your Fleet / balena CLI (latest)** (named, revocable API keys vs 7-day session tokens; `balena login --token` then `Authorization: Bearer`; `balena deploy` uploads images then creates a release entry; per-fleet key listing; “never commit access tokens”): <https://docs.balena.io/learn/deploy/deployment> + <https://docs.balena.io/reference/balena-cli/latest> — validates §1 (named/revocable fleet-scoped keys, §11.4.10 no-commit), §2 (`login --token`, `deploy`), §5 (fleet == project analogue).
- **Expo — Programmatic access (EAS)** (robot/bot accounts that cannot interactively sign in and authenticate only via a token scoped to account-owned resources; `EXPO_TOKEN` env var for CI; treat tokens like passwords, store as CI secrets): <https://docs.expo.dev/accounts/programmatic-access/> — validates §1.1 (account-scoped robot credential, Option-B exchange), §1.2 (`HELIX_OTA_TOKEN` env pattern), §5 (CI-secret incorporation).

- **npm — About access tokens / granular access tokens GA** (least-privilege granular tokens scoped to specific packages/scopes/orgs, auto-expiring, recommended for CI publishing; legacy long-lived tokens removed 2025): <https://docs.npmjs.com/about-access-tokens/>
 - <https://github.blog/changelog/2023-03-21-general-availability-of-granular-access-token-on-npm/> — validates §1.1 (scope-limited key), §1.3 (short-lived/expiring preference, Option B).

Negative finding (§11.4.99): none of the surveyed services stores the cleartext token server-side — all store a hash and show the secret once. This directly supports the `api_keys.key_hash` sketch (`001_initial_schema.up.sql:60`) and §1.1's hash-only rule; no source contradicts the proposed model.

Honest boundary

This is a **design proposal**, not shipped behavior (§11.4.6). Concretely:

- **The object-storage dependency is REAL and unresolved in-repo.** Artifact bytes are validated then discarded today (`handlers_artifact.go:184, 11_* §Honest-gaps 1`). The CLI's authenticate → upload → validate → register → release → deploy path is designable and testable now, but a genuine “device downloads and applies the exact uploaded bytes” e2e **cannot be proven until real object storage lands** (§4). Any claim of full e2e before then would be a §11.4 PASS-bluff; §6 marks that layer SKIP-with-reason.
- **Entity shapes are not decided here.** account, user, membership, and the scoped `api_keys` columns (`account_id/project_id`) are **owned by 20_* (SSOT)**; this doc consumes them and cites the one concrete in-repo sketch (`001_initial_schema.up.sql:57-71`) for the fields that already exist. Where `20_*` finalizes a different shape, `20_*` wins and this doc is reconciled to it.
- **The tenant-aware token claim is a 21_* dependency.** Option B (recommended) needs `account_id+project_id` in Claims (`token.go:31-36` has none today); the auth-model fork is surfaced with tradeoffs (§1.3), not silently chosen for `21_*`.
- **Per-account signing keys, per-scope monotonicity, and a list-artifacts endpoint** are named as required server-side extensions the CLI depends on (§3, §4) but does not itself own; the trust boundary (verify key from server config only, never the request) is explicitly preserved in the proposal.
- **Recommendations carry tradeoffs, not silent decisions** (§11.4.66): Option B over A for auth; per-account signing key with

global fallback; MinIO dev / S3 prod behind a Storage seam; binary + container distribution. Each states the cost of the choice and the condition under which the alternative is correct.